

Review



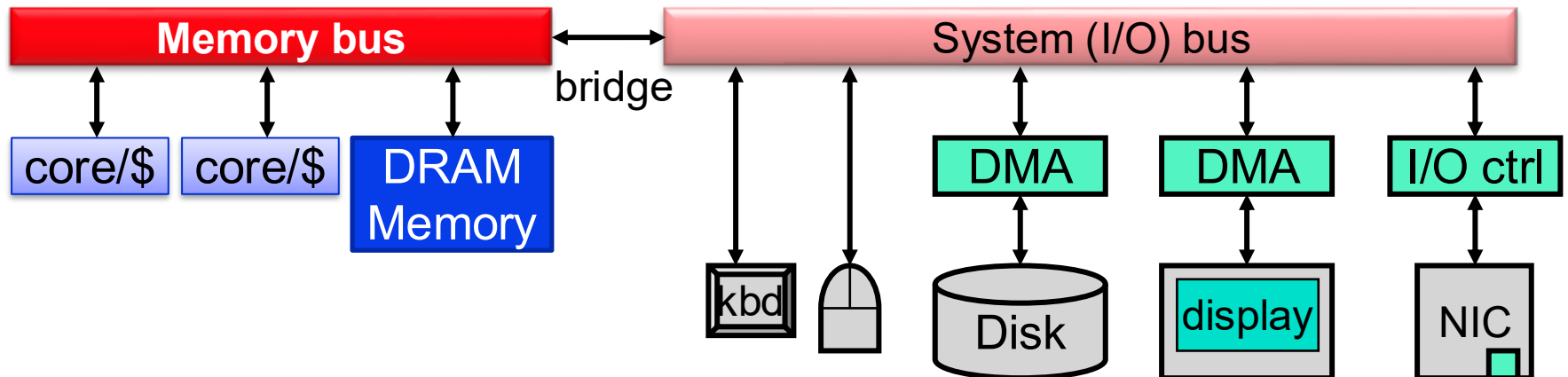
- Last Class:
 - Advanced cache optimizations
 - Advanced translation optimizations
- Today's class:
 - Main memory

Memory technology and optimizations (§ 2.3)

Memory Hierarchy

❑ Cores, caches and **main memory**

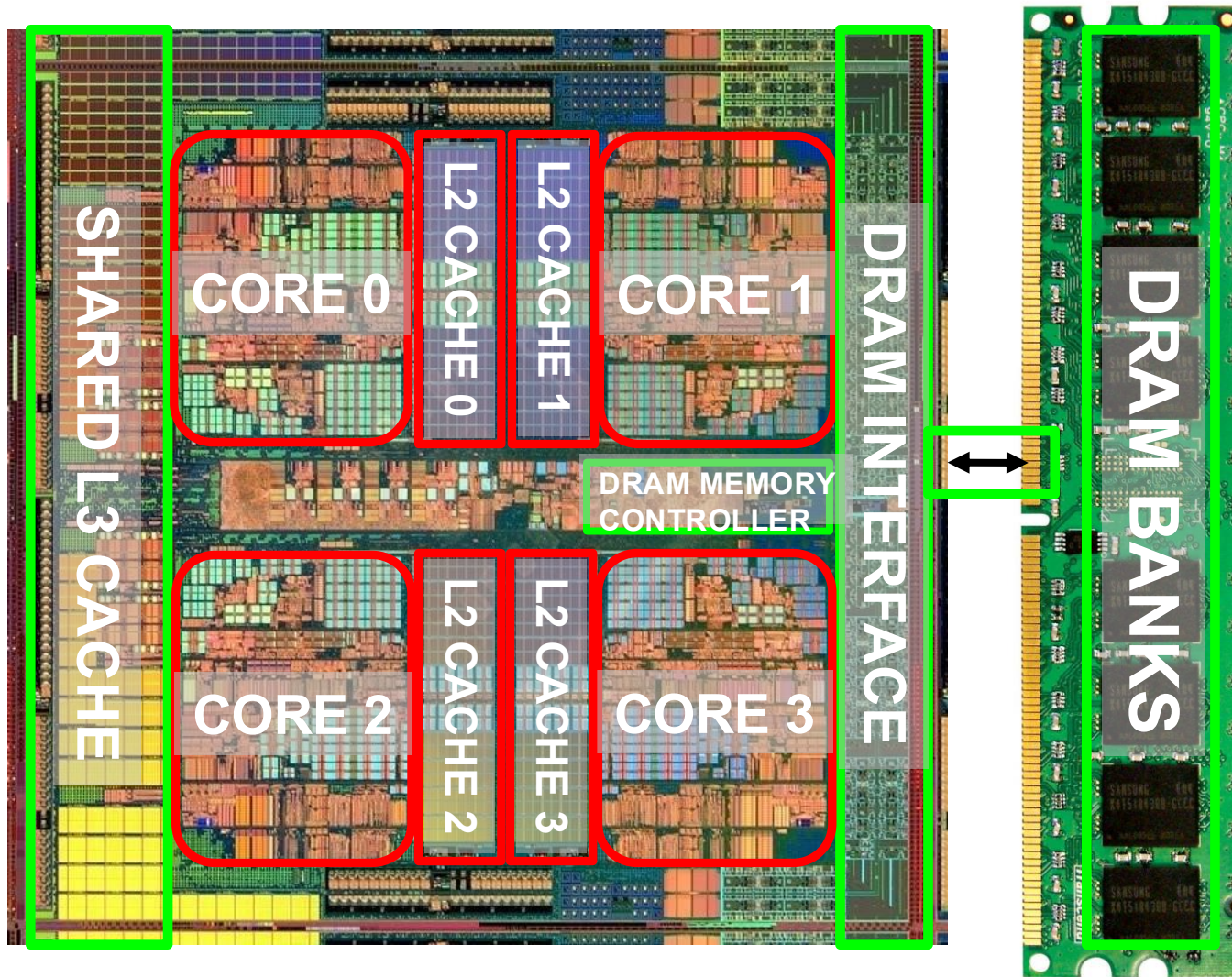
- ❑ Connected by the **memory bus** (aka northbridge, ideally on-chip with the cores and caches)



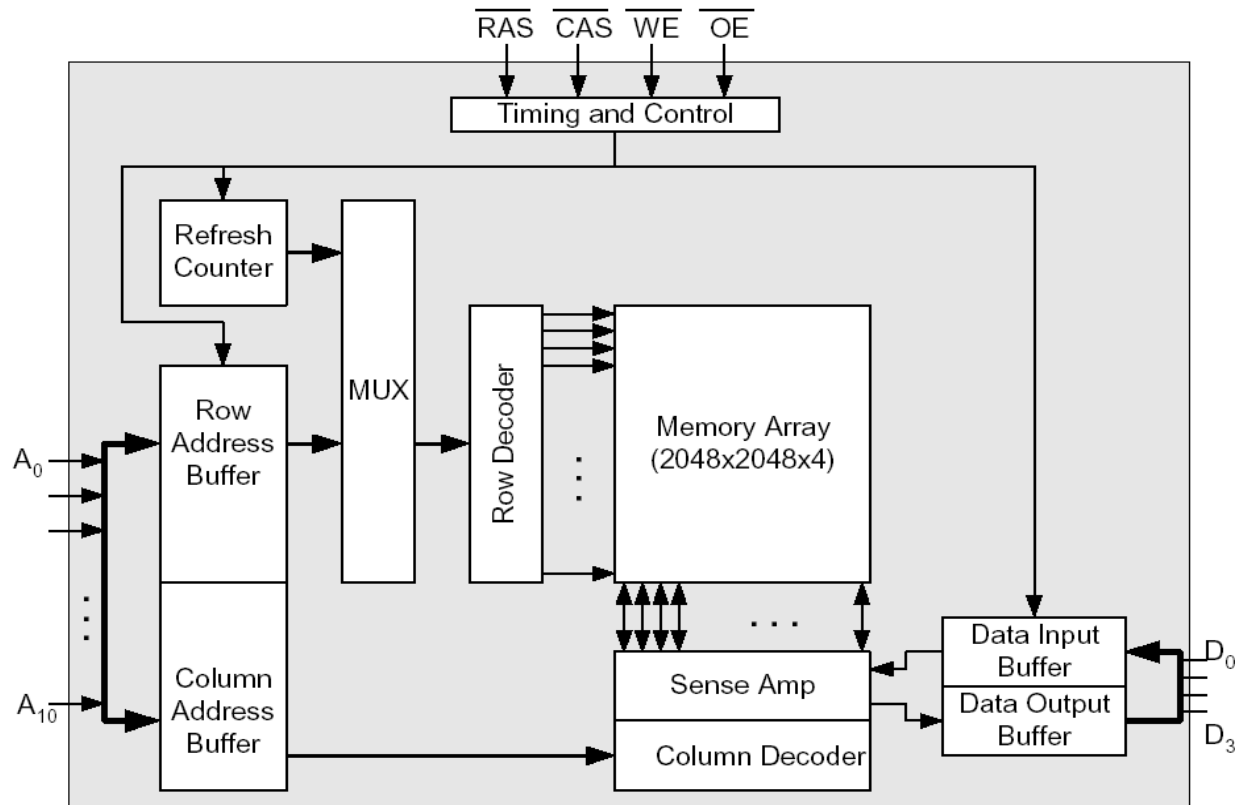
❑ I/O peripherals: storage, input, display, network, ...

- ❑ With separate or built-in DMA
- ❑ Connected by the **system bus** (aka southbridge) which is connected to memory bus

Main Memory in the System



DRAM



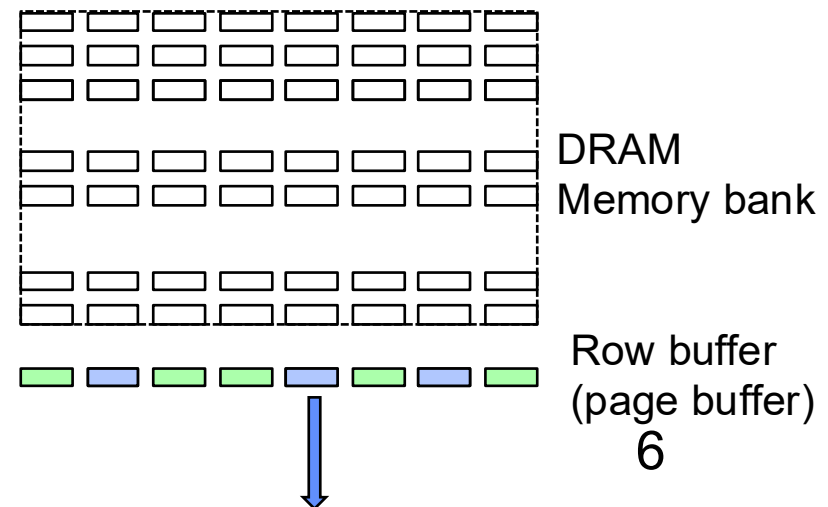
- SDRAM = DRAM with a clocked interface
- DDR SDRAM = double data rate, transfer data at both clock edges
 - DDR2 (1.8 V, 266-400 MHz)
 - DDR3 (1.5 V, 800 MHz)
 - DDR4 (1-1.2 V, 1600 MHz)

DRAM operation

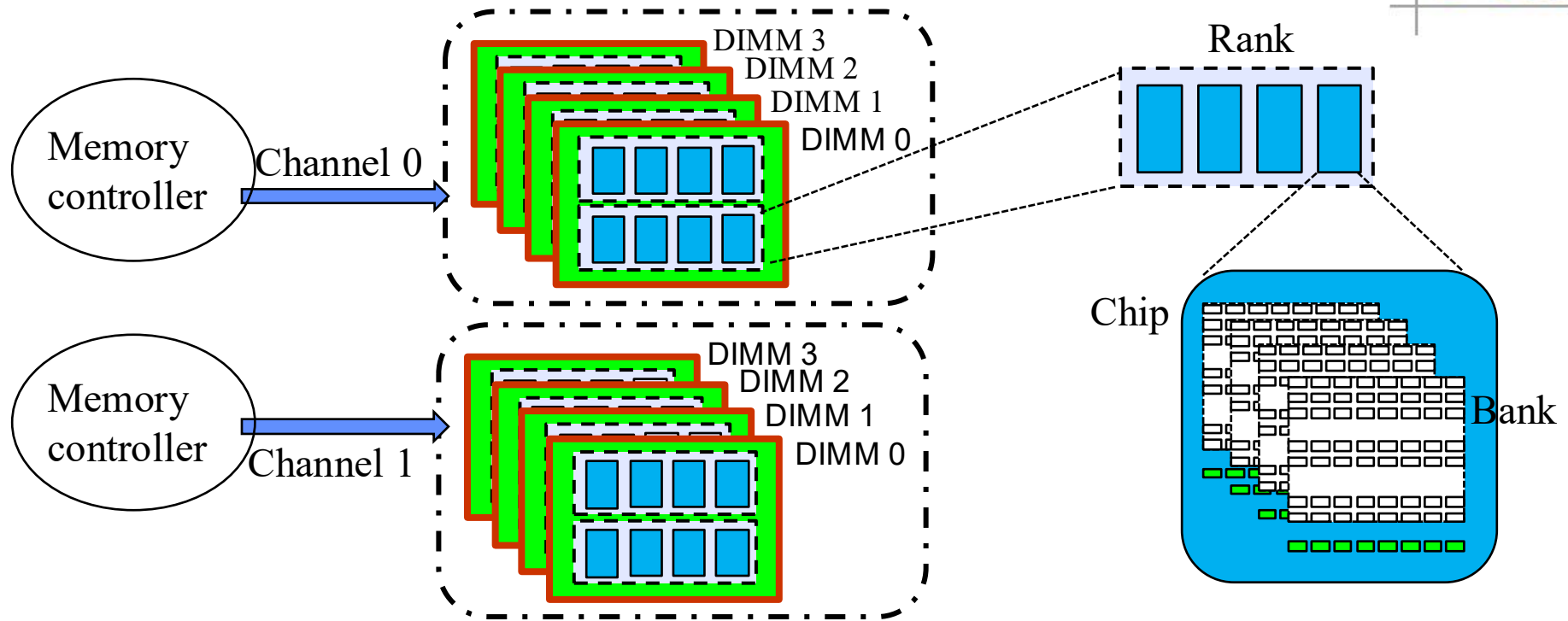


- Each memory bank has a “row buffer”(SRAM registers)
- To read a byte (a similar process applies for writing):
 - The memory controller sends the row address of the byte
 - The entire row is read into the row buffer (the row is opened)
 - The memory controller sends the column address of the byte
 - The memory returns the byte to the controller (from the row buffer)
 - The memory controller sends a Pre-charge signal (close the open row)

- **Closed row/page policy:** close the row after you read the data
- **Open row/page policy:** close the row only when you want to open a new row – useful if you will read again from the same row.



DRAM Organization

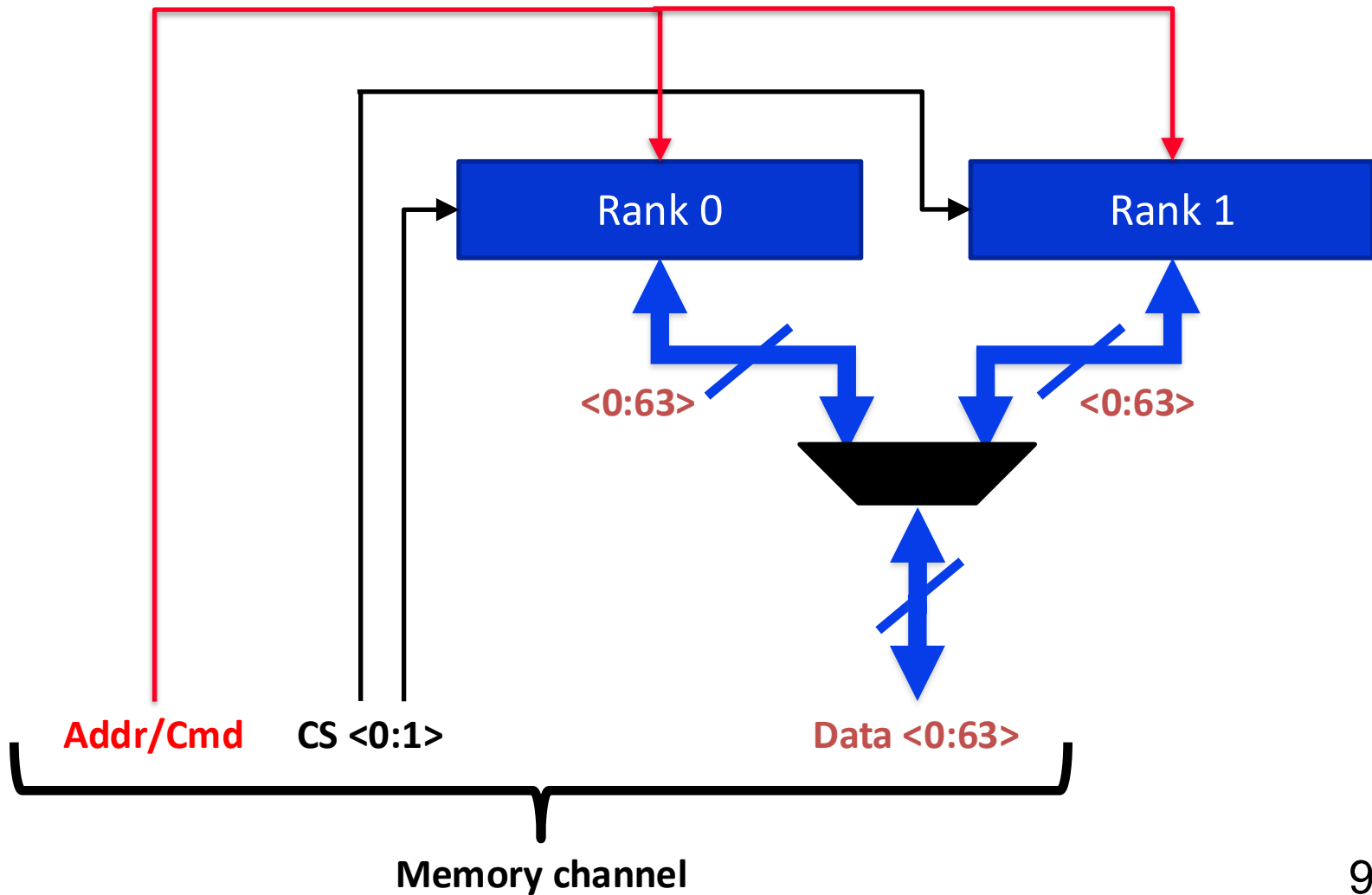


- A memory chip is organized internally as a number of banks (1-8 usually).
- Multiple banks can execute different commands at the same time
- A Rank consists of multiple (parallel) chips contributing to the same transaction. For example, each of 4 chips can provide a byte for a total of 32 data bits (read or written).
- A DIMM (Dual Inline Memory Module) consists of 1 – 4 ranks (2 in the figure) mounted on a single printed-circuit board.
- A Channel supports multiple DIMMS (4 in the above figure)

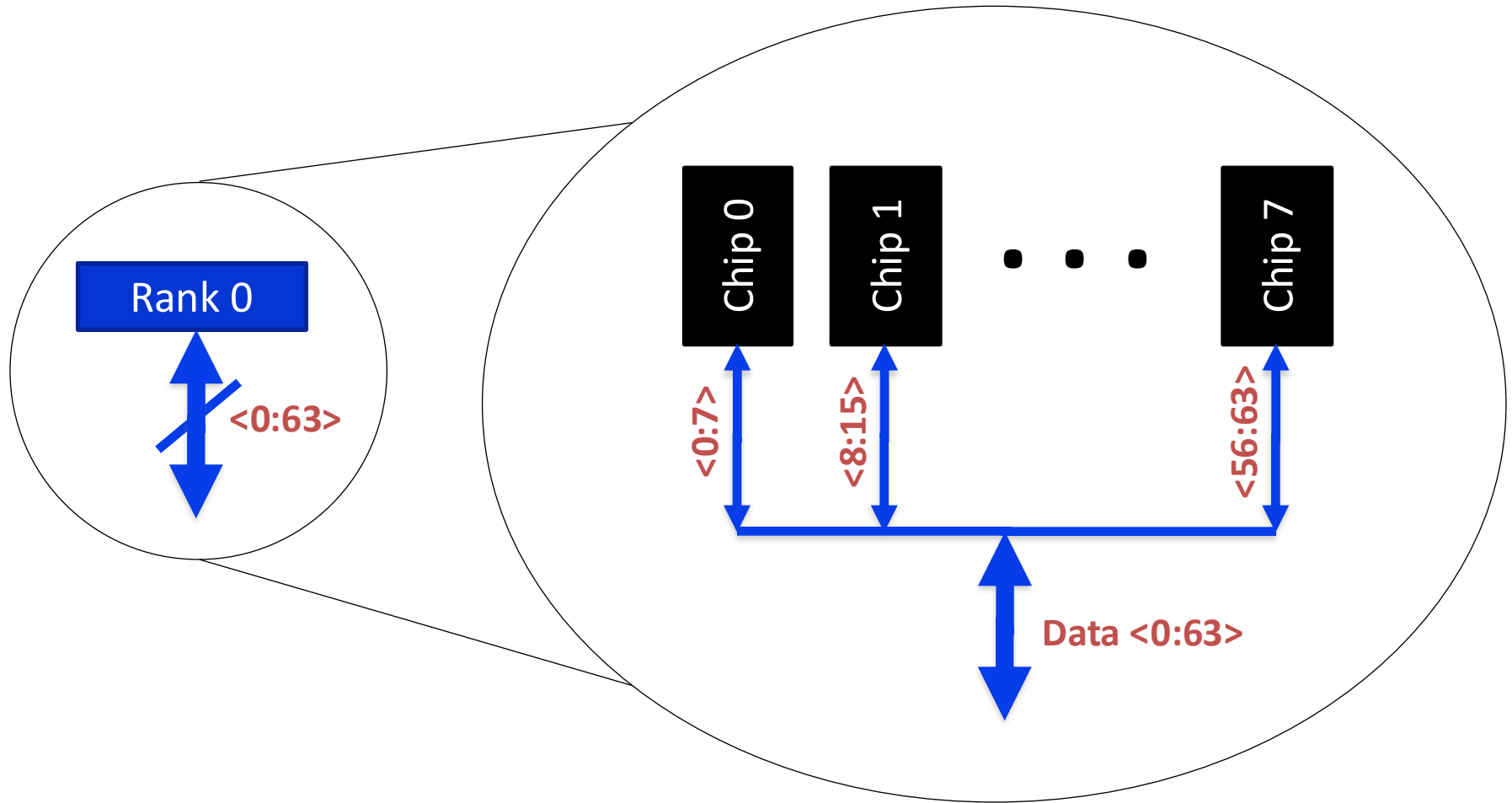
Channels, Memory Controllers, DIMMs, DRAM Chips

- ❑ To maximize memory bandwidth, there are multiple **memory channels** in the system. Each channel is managed by one **memory controller (MC)**, and is connected to one or more **DIMMs**, each containing multiple DRAM chips.
- ❑ These DRAM chips are logically arranged into multiple **ranks**. Internally, each DRAM chip is partitioned into **banks** that are accessed in parallel.
- ❑ In each memory controller, there is one **read queue** and one **write queue** to buffer the outstanding memory requests. A **scheduler** (e.g., FCFS) determines which request should be *serviced* next. For each channel, only one request can be sent to the memory through the bus at each clock cycle.

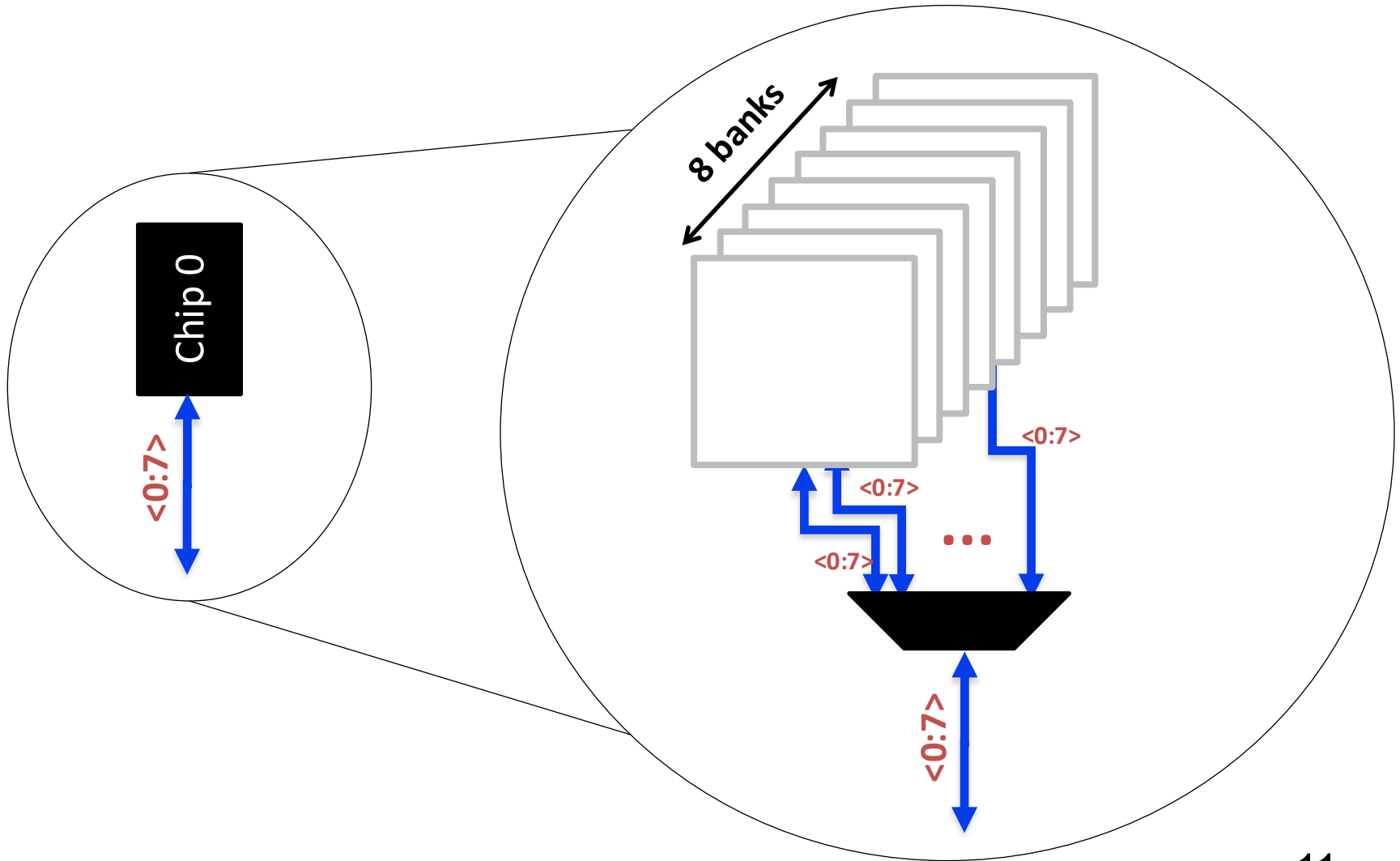
Ranks



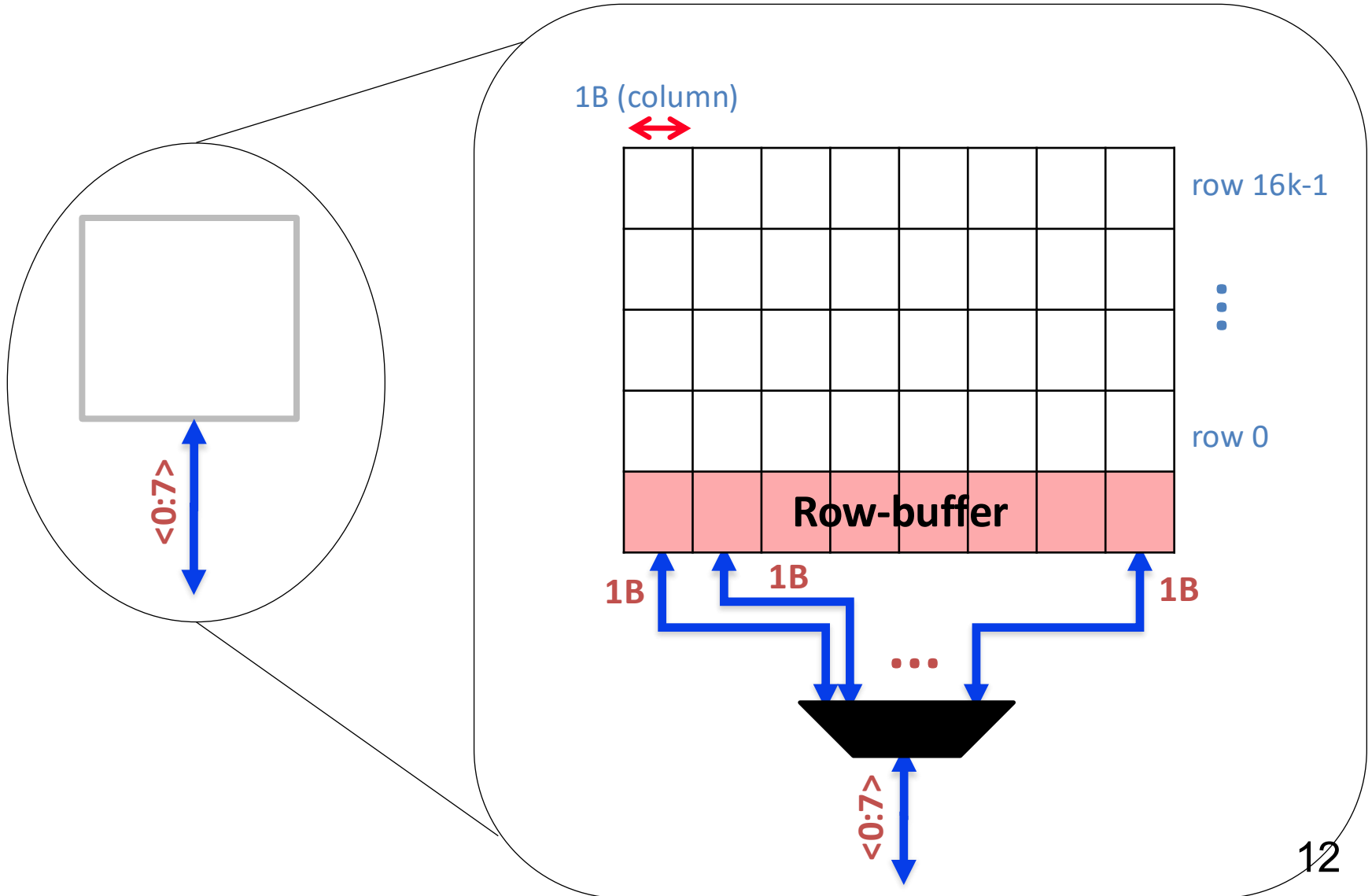
Breaking down a Rank



Breaking down a Chip

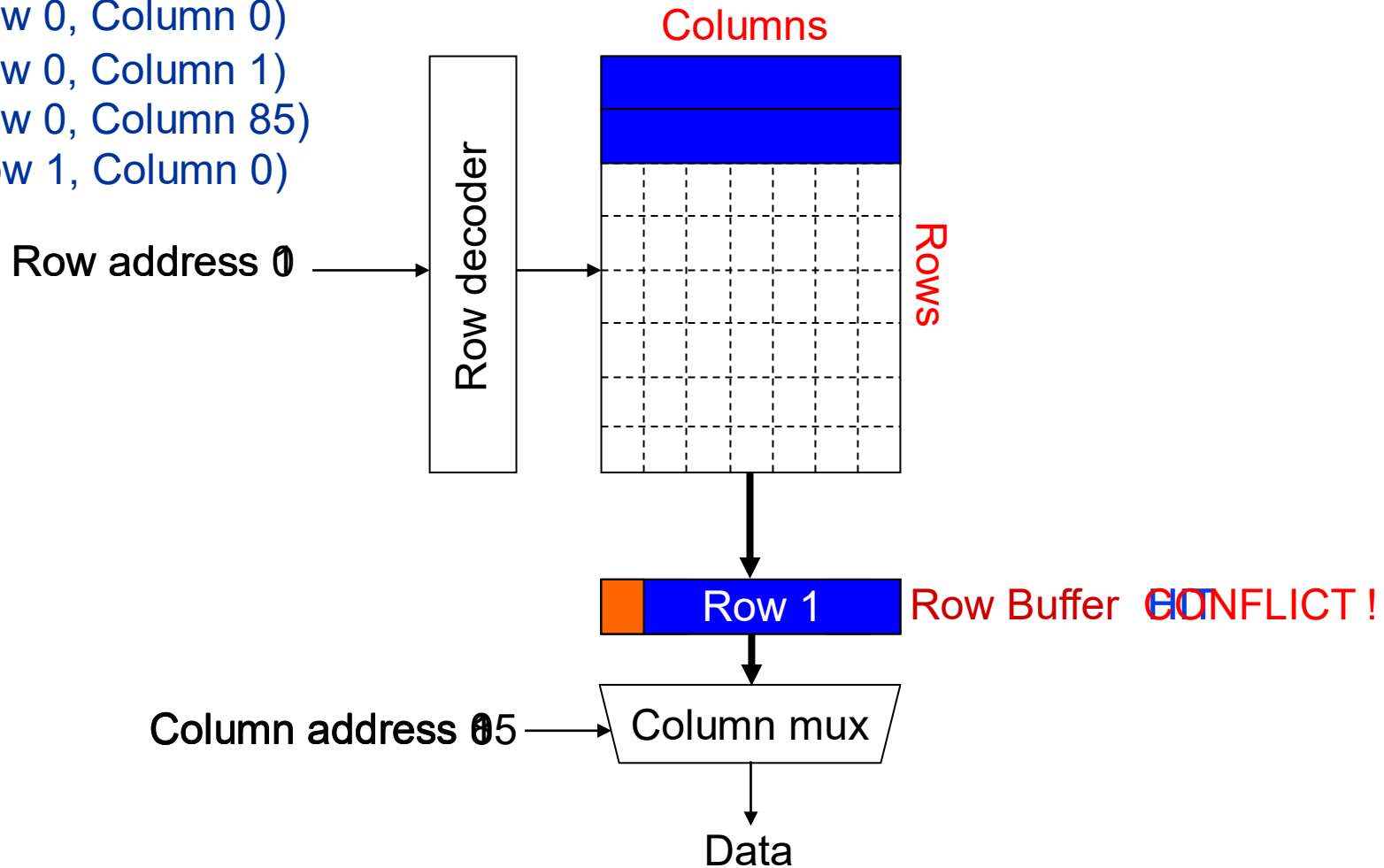


Breaking down a Bank



DRAM Bank Operation

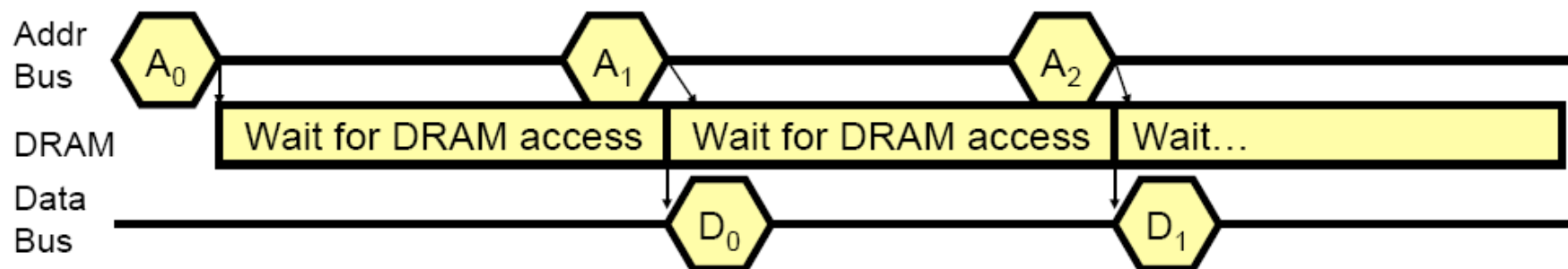
Access Address:
(Row 0, Column 0)
(Row 0, Column 1)
(Row 0, Column 85)
(Row 1, Column 0)



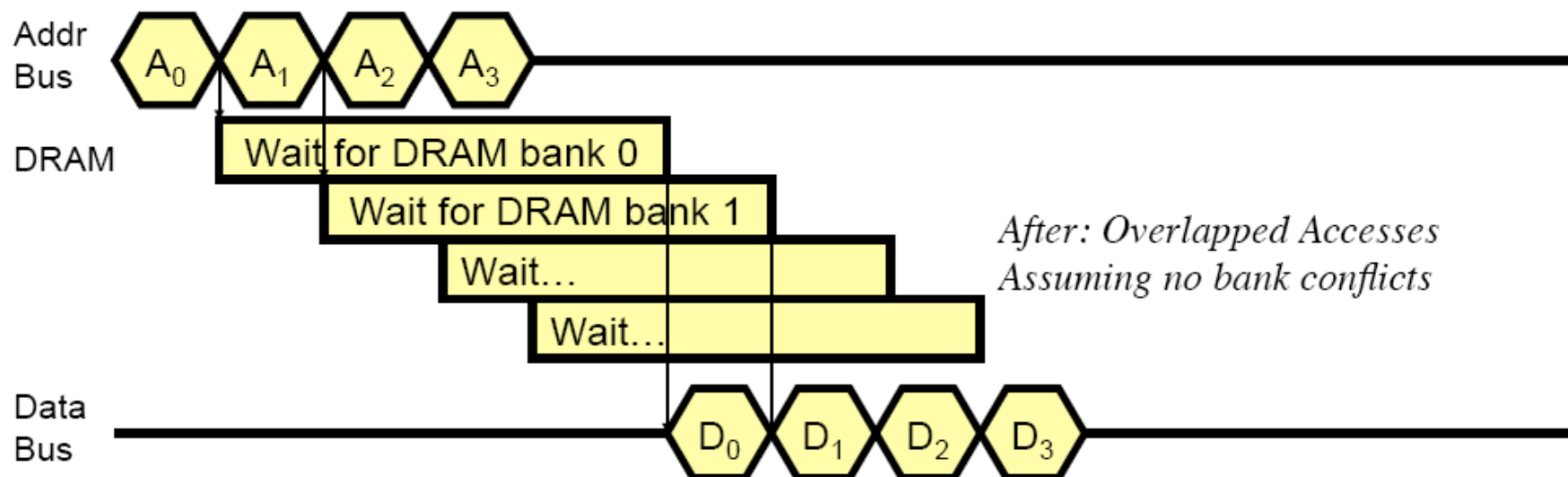
Multiple Banks (Interleaving) and Channels

- ❑ Multiple banks
 - ❑ Enable concurrent (pipelined) DRAM accesses
 - ❑ Data from different banks are NOT obtained in the same cycle
 - ❑ Bits in address determine which bank an address resides in
- ❑ Multiple independent channels serve the same purpose
 - ❑ But they are even better because they have separate data buses
 - ❑ Increased bus bandwidth
- ❑ Enabling more concurrency requires reducing
 - ❑ Bank conflicts
 - ❑ Channel conflicts

How Multiple Banks/Channels Help



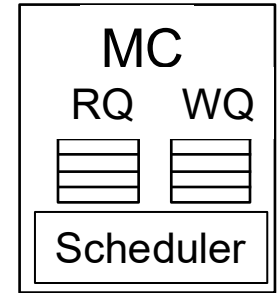
*Before: No Overlapping
Assuming accesses to different DRAM rows*



*After: Overlapped Accesses
Assuming no bank conflicts*

On-Chip Memory Controller

- ❑ A **front-end** buffers requests to the DRAM from the LLC (is independent of DRAM type)
 - ❑ Decodes the address into bank, row, column
 - ❑ Schedules requests to DRAM for max bandwidth
 1. Reorders bursts to minimize bank conflicts
 2. Prioritizes reads over writes (a core is waiting for the read data)
 - ❑ Sends responses from the DRAM back to the LLC



Memory Controllers

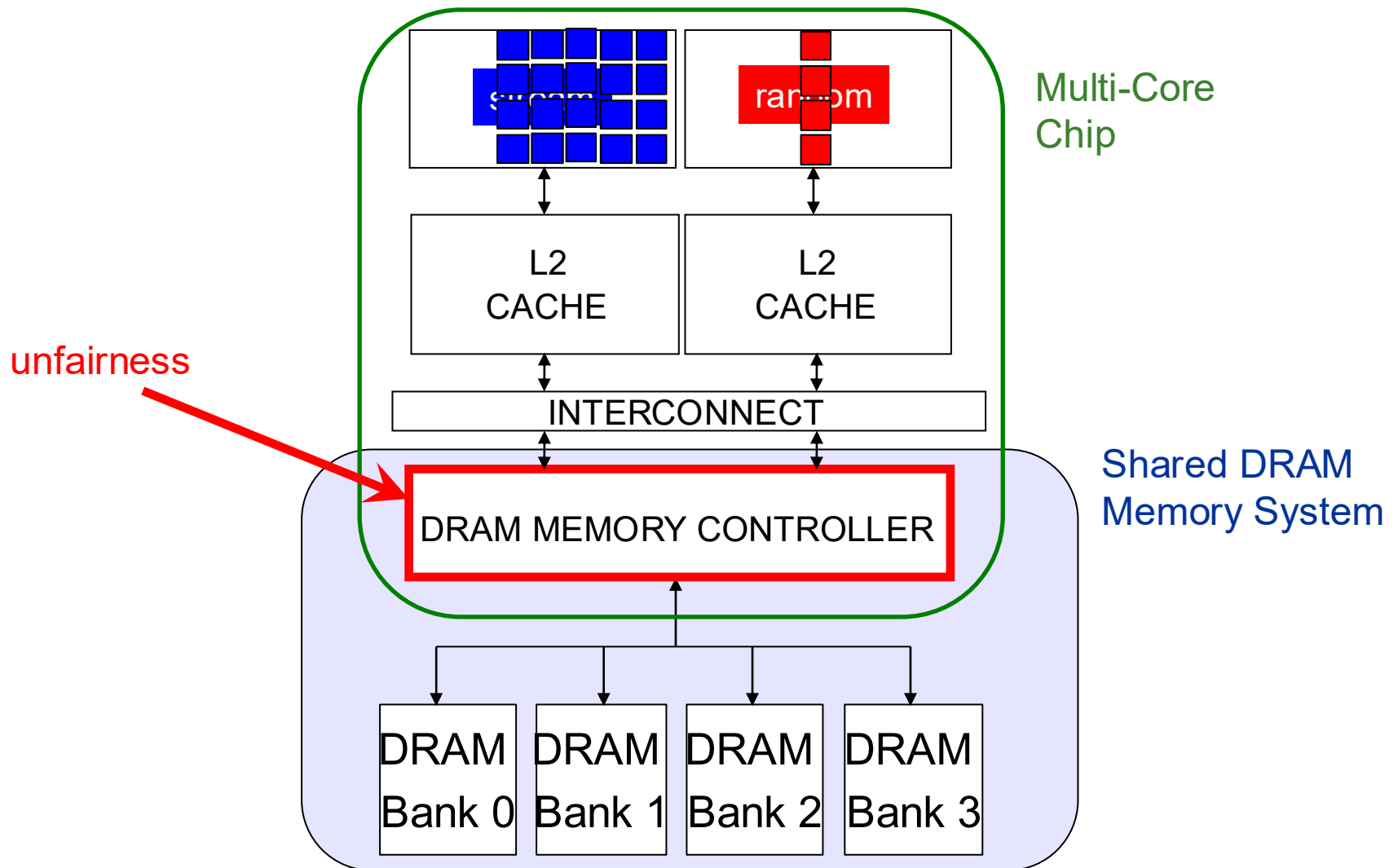
- ❑ A row-conflict memory access takes significantly longer than a row-hit access
- ❑ Current controllers take advantage of this fact
- ❑ Commonly used scheduling policy (FR-FCFS) [Rixner 2000]*
 - (1) **Row-hit first**: Service row-hit memory accesses first
 - (2) **Oldest-first**: Then service older accesses first
- ❑ This scheduling policy aims to maximize DRAM throughput
- ❑ Explore row reuses [Kandemir 2015]*
- ❑ Concurrency-aware scheduling in GPUs [Jog et al. ASPLOS 2013]

*Rixner et al., “Memory Access Scheduling,” ISCA 2000.

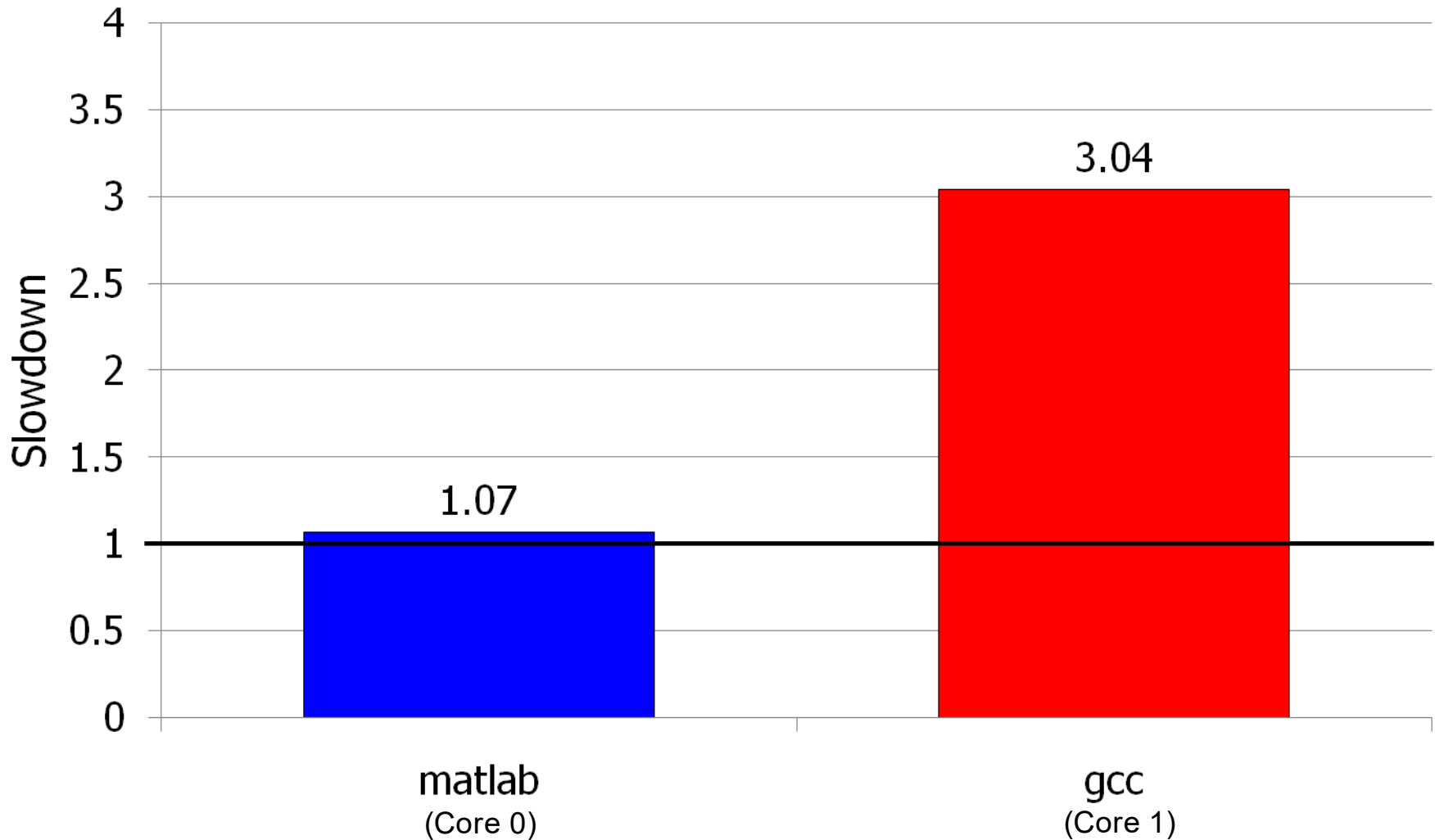
*Zuravleff and Robinson, “Controller for a synchronous DRAM ...,” US Patent 5,630,096, May 1997.

*Mahmut Kandemir, Hui Zhao, Xulong Tang, and Mustafa Karakoy. 2015. “Memory Row Reuse Distance and its Role in Optimizing Application Performance”. SIGMETRICS '15.

Uncontrolled Interference: An Example



Slowdown in Multi-Core Systems



Moscibroda and Mutlu, “[Memory performance attacks: Denial of memory service in multi-core systems](#),” USENIX Security 2007.

A Memory Performance Hog

```
// initialize large arrays A, B  
for (j=0; j<N; j++) {  
    index = j*linesize; streaming  
    A[index] = B[index];  
    ...  
}
```

STREAM

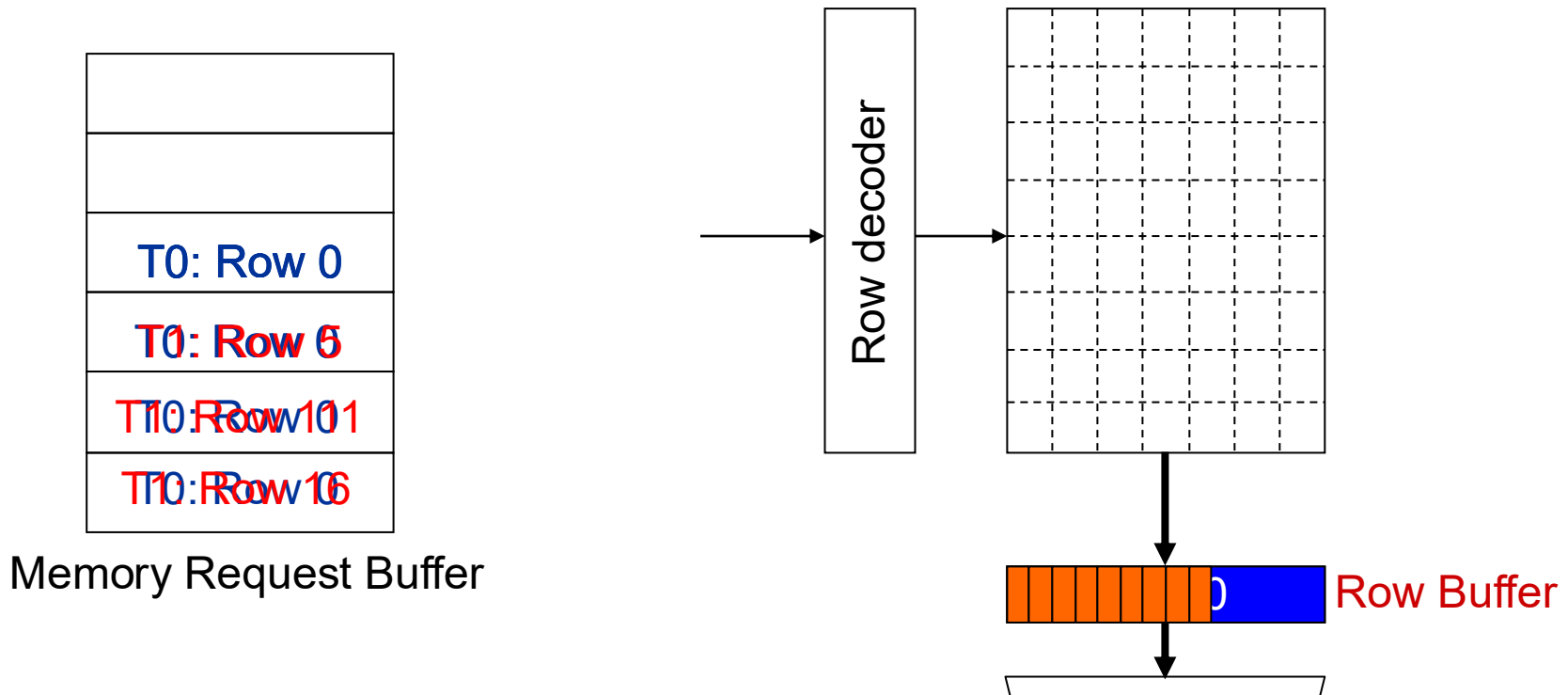
- Sequential memory access
- Very high row buffer locality (96% hit rate)
- Memory intensive

```
// initialize large arrays A, B  
for (j=0; j<N; j++) {  
    index = rand(); random  
    A[index] = B[index];  
    ...  
}
```

RANDOM

- Random memory access
- Very low row buffer locality (3% hit rate)
- Similarly memory intensive

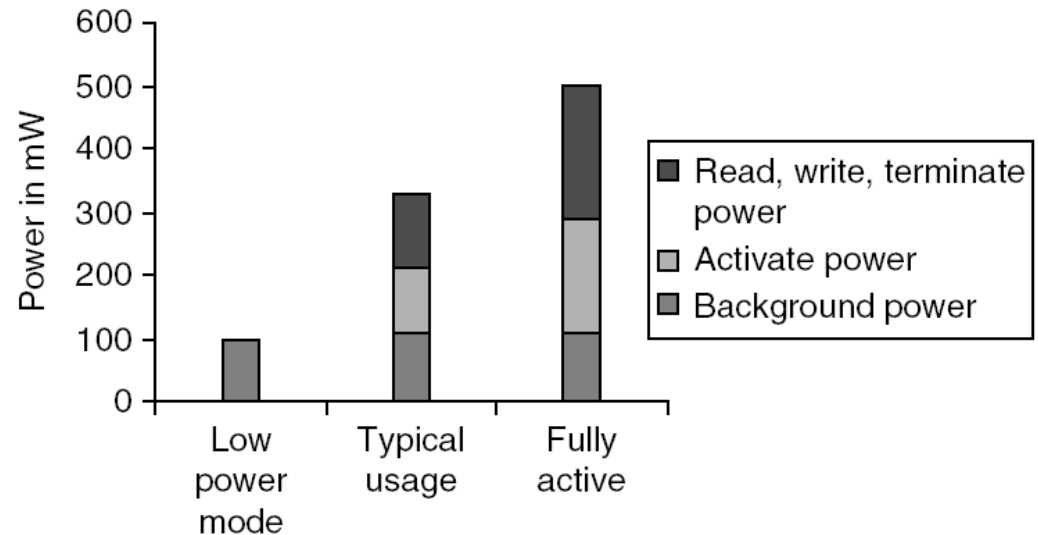
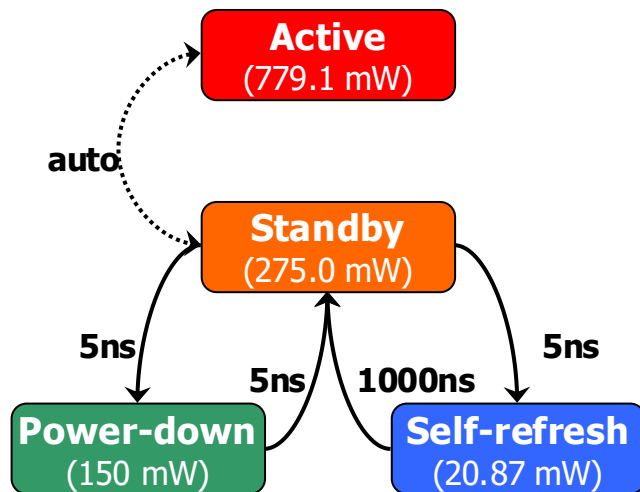
What Does the Memory Hog Do?



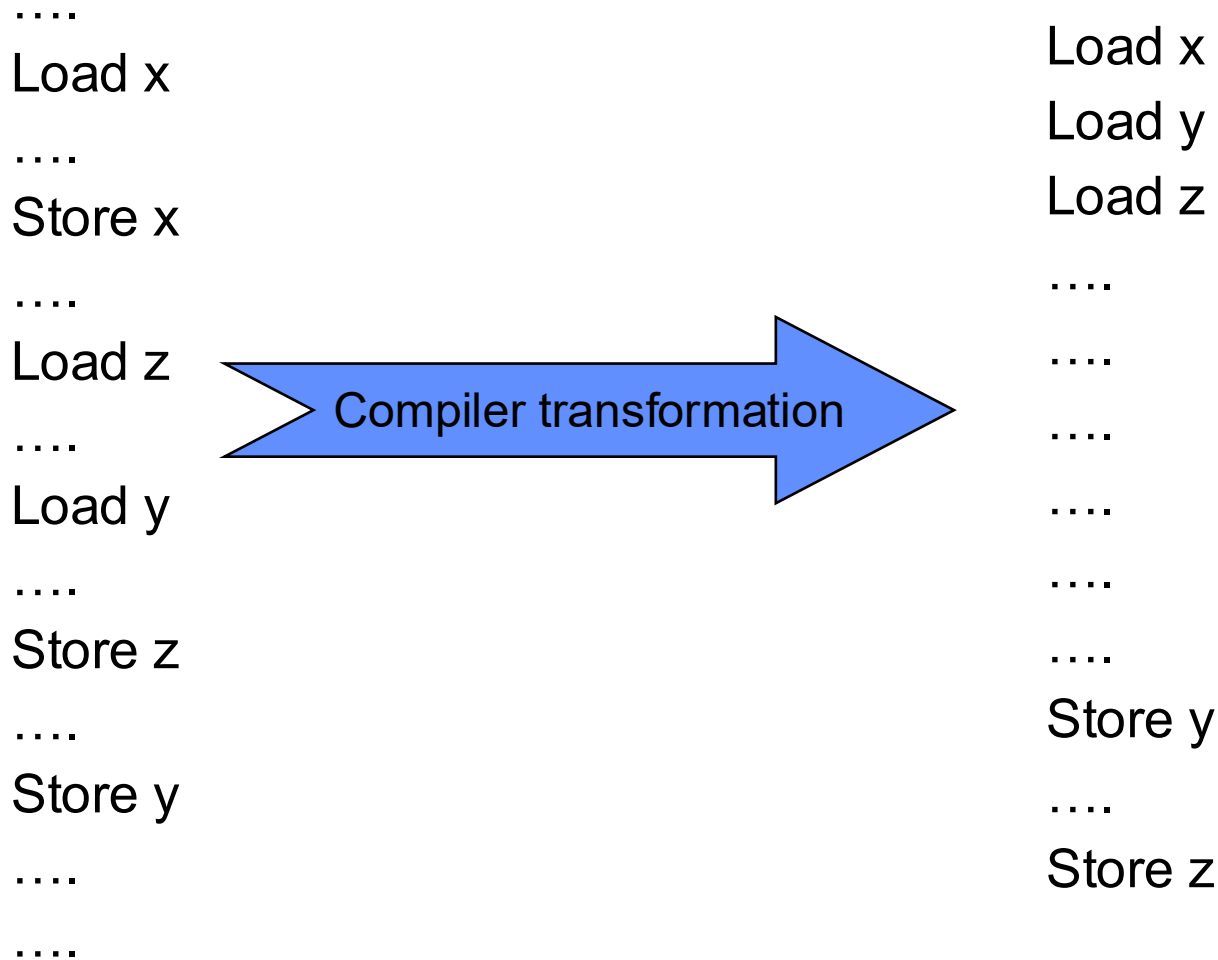
Row size: 8KB, cache block size: 64B
128 (8KB/64B) requests of T0 serviced before T1

Reducing memory power consumption

- Lower voltage
- Low power mode (ignores clock, continues to refresh)



Example of compiler assisted Memory Power Management?



Code transformations to increase the memory idle time (the time between memory accesses).

Example of compiler assisted Memory Power Management?



Declare A[], B[], C[], D[]

....

Access A

....

Access D

....

Access B

....

Access C

....

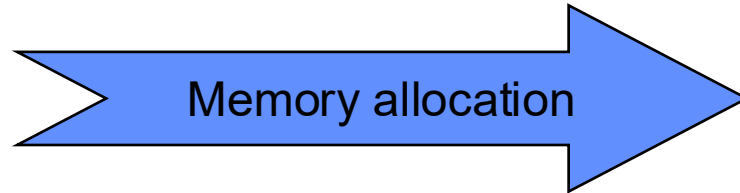
Access B

....

....

A[], B[]

C[], D[]



OR

A[], D[]

C[], B[]

Algorithms that use the access pattern to allocate memory to banks in a way that maximizes bank idle times

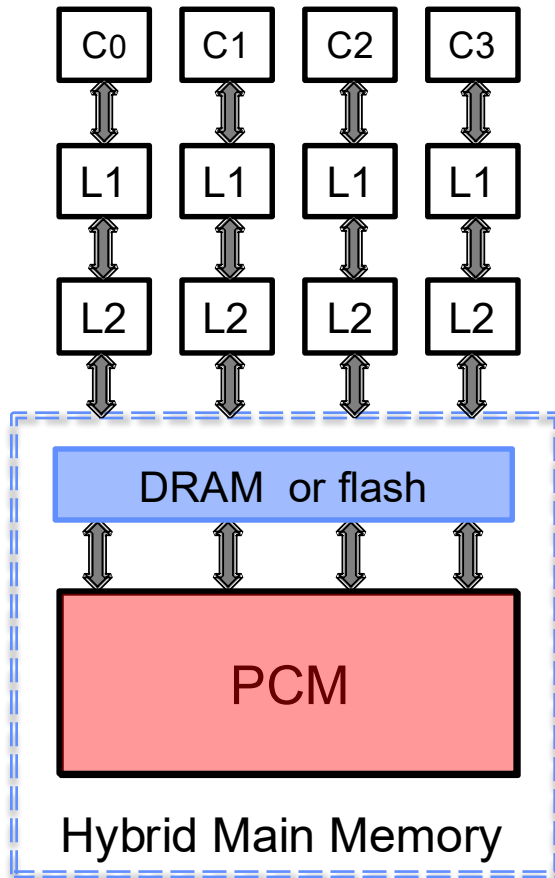
Alternative memory technology



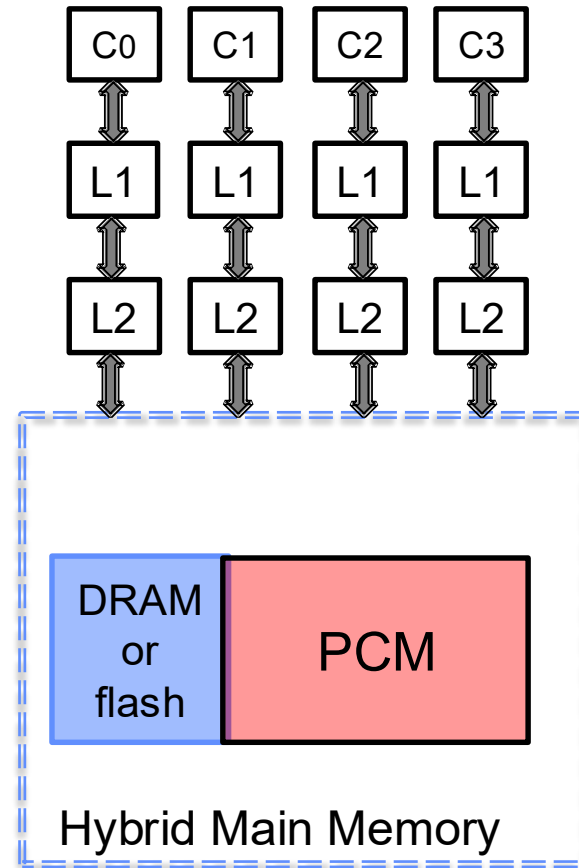
	Read Speed	Write Speed	Cell Area	Endurance	Addressability
DRAM	20~50ns	20~50ns	$6F^2$	10^{15}	Yes
SRAM	~2ns	~2ns	$146F^2$	$10^{15} \sim 10^{16}$	Yes
NAND Flash	25us	500us	$5F^2$	$10^4 \sim 10^5$	No
STT-RAM	2ns	10ns	$37 \sim 40F^2$	10^{12}	Yes
PCM	30~50ns	~1us	$5 \sim 8F^2$	$10^7 \sim 10^8$	Yes
Domain wall memory					

Hybrid main memory

- To deal with asymmetric read and write
 - Write is slower, more time consuming and causes wear



Vertical organization



Horizontal organization

Intel Optane Technology

	DRAM	Intel Optane	Flash Memory (SSD)
Speed	Very Fast	Slower than DRAM, but much faster than flash memory	Slower than both DRAM and Intel Optane
Cost	Expensive	Costs less than DRAM but more than flash memory	Affordable
Volatile / Non-Volatile	Volatile	Non-Volatile	Non-Volatile
Latency	Low	Low	High
Reliability	High	Excellent read response times compared to flash-based drives	Low
Endurance	High	High	Low